

SDR-Based DDR Air-Gap Data Exfiltration

Nikas Ioannis Iason

*Electrical and Computer Engineering
University of Thessaly
Volos, Greece
ioannikas@uth.gr*

Tsiantos Dimitrios

*Electrical and Computer Engineering
University of Thessaly
Volos, Greece
dtsiantos@uth.gr*

Tsogkas Panagiotis Nikolaos

*Electrical and Computer Engineering
University of Thessaly
Volos, Greece
ptsogkas@uth.gr*

Abstract—Air-gapped computer systems are widely used to protect critical infrastructure from network-based cyber threats, yet they remain vulnerable to electromagnetic side-channel attacks. This report details the implementation and analysis of a side-channel attack exploiting memory bus emissions, demonstrating how radio signals generated by a computer’s RAM can be exploited to exfiltrate data without physical or network connections. The transmitter was developed using a program (malware) that manipulates memory traffic on the DDR bus by utilizing memory access patterns that bypass CPU caches. We use an On-Off Keying modulation method where memory-intensive bursts represent logic ‘1’s and idle periods represent logic ‘0’s to transmit a payload of bytes. For the receiver, we utilized a Software Defined Radio that demodulates the signal to recover the exfiltrated data.

Index Terms—Air-gap, Side-channel attack, Electromagnetic interference, DDR SDRAM, Software-defined radio, On-Off Keying, Data exfiltration

I. INTRODUCTION

A. Air-Gapped Security

In cybersecurity, air-gapped systems are often considered the ultimate defensive measure. An air-gapped system is one that is physically isolated from networks, including the public Internet and local area networks. By removing all network interface controllers security architects aim to create an unbreakable barrier against remote exfiltration. These systems are typically reserved for the most sensitive environments, such as military command centers, critical infrastructure, and financial institutions, where the confidentiality and integrity of data are critical. The assumption is that if a computer cannot physically route packets to the outside world, the data contained within it is secure from remote theft.

B. Electromagnetic Emissions

However, the assumption that physical isolation equates to electronic silence is fundamentally flawed. Any electronic device that processes information relies on the flow of electric current. A variable electric current in time, emits electromagnetic radiation due to the magnetic field generated. While these emissions are typically viewed as unintentional noise or interference, they can serve as a covert communication channel for a knowledgeable adversary. This class of vulnerability, known as a side-channel attack, exploits the physical implementation of a computer system rather than weaknesses in its software algorithms. Over the decades, researchers have demonstrated

that data can be exfiltrated from air-gapped systems using various unintended carriers, including acoustic noise from fans, thermal patterns from CPU heat, and optical signals from hard drive LEDs [1], [2], [3].

C. Memory Bus Attack Vector

Among these side channels, the RAM bus presents a potential vector. Modern computers utilize Double Data Rate memory, which operates at extremely high frequencies and requires rapid switching of electrical signals to transfer data between the CPU and memory modules. This high-speed switching generates significant electromagnetic interference. By manipulating the patterns of memory access, specifically by forcing the CPU to read and write large blocks of data in specific sequences, malware can intentionally modulate this EMI, turning the memory bus into a “radio transmitter”.

D. Project Structure

The objective of this research project was to practically implement and analyze the memory bus electromagnetic emission attack vector in a controlled environment. We tried to demonstrate that a user-space application, requiring no special privileges or modified hardware on the victim machine, could successfully exfiltrate data across an air gap to a nearby receiver. Our project consists of two main components:

The transmitter: We developed a program that acts as a malware and utilizes memory operations that generate high-intensity electromagnetic interference in bursts. By carefully selecting memory address to perform operations on we bypass the CPU caching mechanism and we can keep the main memory bus active which results in detectable electromagnetic signals.

The receiver: We constructed a software-defined radio (SDR) receiver using a simple ADALM-PLUTO development board and a GNU Radio flow. The receiver captures signal at the frequency the bus emits electromagnetic signals, it removes the noise, and applies an On-Off keying (OOK) demodulation scheme to reconstruct the binary data.

This report details the technical implementation of both the transmitter and receiver architectures, analyzes the signal processing challenges encountered in distinguishing the target signal from environmental background noise, and presents experimental results demonstrating the feasibility of data ex-

filtration at various bitrates and physical distances from the target.

II. BACKGROUND AND RELATED WORK

A. Electromagnetic Side-Channel Attacks

The field of electromagnetic (EM) side-channel analysis relies on the fundamental principle that electronic devices inevitably leak unintended information through physical emanations. While cryptographic algorithms may be mathematically secure, the physical hardware implementing them—whether a CPU, a memory bus, or a peripheral cable—is imperfect. These devices emit radio frequency (RF), acoustic, and thermal signals that correlate with the data being processed, allowing an adversary to reconstruct sensitive information without physical intrusion.

The study of compromising emanations dates back to the mid-20th century, primarily within military and intelligence communities under the codename TEMPEST. Initially, this research focused on the leakage from electromechanical encryption devices and teleprinters. By the 1960s, it was understood that sharp current pulses driving teleprinter magnets generated RF spikes that could be received and decoded nearby. Public awareness of these vulnerabilities emerged significantly with the publication of research on the security risks of computer monitors in 1985 [4]. This work demonstrated that the video signal driving a Cathode Ray Tube (CRT) display could be reconstructed from a distance using a standard television receiver modified with a manually tuned oscillator. This phenomenon, often called “Van Eck Phreaking” exploited the high-energy signals required to steer the electron beam in CRTs, which acted as unintentional broadcast antennas [4] [5].

Subsequent research conducted during the late 1990s and early 2000s broadened the scope of this field by probing the limitations of hardware shielding and establishing that software could be utilized to intentionally modulate emissions. These investigations showed that digital interfaces were not immune to surveillance, revealing that they leaked recoverable data via high-frequency electromagnetic noise [6]. Contemporary studies have further escalated this threat significance by applying deep learning algorithms to reconstruct screen imagery from increasingly faint signals [7].

In recent years, research has moved beyond video output to internal data buses and power lines. It has been demonstrated that USB cables can leak timing and data information. Because USB communication relies on differential signaling, ideal cables should cancel out EM radiation. However, manufacturing imperfections and impedance mismatches often result in “common-mode” currents, creating measurable leakage that correlates with keystrokes or file transfers [8].

Information leakage is not limited to air-gapped RF. The fluctuations in a CPU’s power consumption during cryptographic operations create noise on the power lines. This technique, known as Power Analysis (comprising Simple Power Analysis - SPA, and Differential Power Analysis - DPA), allows attackers to extract cryptographic keys by monitoring the main power cable of a device. Recent studies have successfully

extracted AES keys using both EM and power line side-channels, demonstrating that the voltage fluctuations caused by the CPU’s voltage regulator can be picked up by nearby radio equipment [9].

B. Air-Gapped Networks and Covert Channels

To put into perspective the severity of electromagnetic leakage, it is essential to understand the defensive posture of air-gapped networks. By physically isolating systems from unsecured networks, these measures aim to eliminate all logical pathways for remote connection. In such environments, standard network-based exploitation techniques are rendered ineffective, as there is no route for an external attacker to reach the target.

However, security researchers have demonstrated that air-gaps are not impermeable. The threat model typically assumes that an adversary has already bridged the “air gap” to infect the target machine—perhaps via a supply chain interdiction or a malicious insider using infected removable media (USB). Once the malware is installed, the challenge now becomes to data exfiltration. Since the machine has no network card connected, the malware must establish a covert channel.

It is important to distinguish between a side channel and a covert channel. A traditional side channel (e.g. Power Analysis) is passive; the attacker monitors the victim’s natural operations to infer secrets. In contrast, the attacks described in this report function as covert channels. In this case, the malware controls the system and makes it produce electromagnetic signals, turning the system into a transmitter. This difference is critical because covert channels allow for the transmission of arbitrary data rather than just the passive leakage of signals.

C. DDR Memory Architecture

In order to understand how Random Access Memory (RAM) can leak information by emitting electromagnetic signals we need to look at the physical layer of data transmission. Modern systems use Double Data Rate (DDR) SDRAM, which improves performance by transferring data on both the rising and falling edge of the clock. This way the bandwidth is effectively doubled compared to older Single Data Rate (SDR) memories, without increasing the clock frequency [10].

The physical mechanism behind this leakage is described by Maxwell’s Equations, specifically the relationship between changing electric currents and magnetic fields. Ampère’s Law with Maxwell’s addition can be expressed as:

$$\nabla \times \mathbf{B} = \mu_0 \left(\mathbf{J} + \epsilon_0 \frac{\partial \mathbf{E}}{\partial t} \right)$$

Where $\mathbf{B}$ represents the magnetic field, $\mathbf{J}$ is the current density, and $\frac{\partial \mathbf{E}}{\partial t}$ is the changing electric field. In the context of a computer motherboard, the data bus traces act as conductors carrying rapid pulses of current ($\mathbf{J}$). When the CPU writes to or reads from RAM, the current along these traces changes rapidly. According to the equation above, this time-varying current inevitably generates a corresponding time-varying magnetic field ($\mathbf{B}$) that radiates outward from the motherboard.

The strength and detectability of these signals are amplified by the high frequencies at which modern memory operates. DDR3 and DDR4 modules run at very high clock speeds: for instance, a standard DDR3-1333 specification involves an I/O bus clock of 667 MHz, which effectively transfers data at 1333 MT/s (MegaTransfers per second). Because digital signals are square waves, they generate significant energy not just at the fundamental frequency, but at odd integer harmonics (3rd, 5th, 7th, etc.). The capture files used in this research exhibit a center frequency of 1.333 GHz. This frequency is not arbitrary, it corresponds directly to the harmonics of the DDR clock or the fundamental frequency of the data transfer rate, confirming that the radio equipment is tuning into the emanations generated specifically by the memory bus activity [10].

To weaponize these emissions, the leakage mechanism is typically manipulated using On-Off Keying (OOK) modulation. By programmatically alternating the CPU's workload between memory-intensive operations (which generate high EM noise) and idle loops (which generate minimal noise), the amplitude of the emanated carrier wave can be controlled. In the spectral domain, this modulation manifests as the distinct presence or absence of the target frequency, effectively enabling the transmission of binary data across an air gap.

A significant challenge in generating consistent electromagnetic emissions from RAM is the interference of the CPU's cache hierarchy. Modern processors are designed to hide the latency of RAM accesses by storing more frequently accessed data in multiple levels of cache hierarchy. If the code responsible for generating the signal were to operate solely within these caches, the data traffic would remain internal to the CPU, leaving the external memory bus silent and rendering the side channel ineffective. This limitation is later bypassed through the use of Non-Temporal store instructions.

D. Software Defined Radio

The interception and analysis of these memory emanations are facilitated by Software Defined Radio (SDR) technology. Unlike traditional hardware radios, where signal processing components like mixers, filters, amplifiers, and modulators are implemented in fixed analog circuits, an SDR implements these components in software. This allows for a highly reconfigurable radio system where parameters can be changed on the fly without physical modifications.

The specific device used in this study, the ADALM-PLUTO, acts as a flexible radio front-end. It captures the analog RF signals from the environment, digitizes them using an Analog-to-Digital Converter (ADC), and streams the raw In-phase and Quadrature (I/Q) data to a host computer via USB. The PlutoSDR is based on the AD9363 RF transceiver, which operates from 325 MHz to 3.8 GHz, covering the wide range of frequencies required to detect memory bus harmonics [11].

The use of SDR offers distinct advantages for side-channel research over conventional spectrum analyzers. The reconfigurability of the platform allows the receiver to listen to different memory bus harmonics dynamically. If the primary

frequency is obscured by environmental noise (such as Wi-Fi or other technologies), the software can instantly retune to the third harmonic or a sideband without hardware adjustments.

Second, the availability of raw I/Q data enables advanced post-processing. Once the data is received by the host computer, tools like GNU Radio can be used to construct a complex signal processing flow graph. This typically involves a low-pass filter to isolate the carrier wave, followed by a magnitude calculation block (AM demodulation) to recover the On-Off Keying envelope. This software-based approach allows for the application of digital filters with extremely sharp rolloffs, enabling the isolation of memory leakage signals that would otherwise be lost in the noise floor of a standard hardware receiver.

III. THREAT MODEL

A. Attacker Capabilities

The premise of this attack relies on a pre-existing compromise of the air-gapped system. We assume a capable adversary has utilized established vectors—such as supply chain interdiction or the introduction of infected removable media via an unwitting insider—to bypass physical isolation measures.

Once the system is compromised, the malware operates under strict operational constraints to ensure persistence and stealth. The attack is designed to execute entirely in user space, utilizing standard libraries and intrinsic functions without requiring kernel-level privileges or root access. Furthermore, the model assumes no physical modifications to the victim machine. Throughout this process, the execution must remain lightweight, avoiding noticeable performance degradation that might alert the user. Under these constraints, the attacker's primary objective shifts to maximizing the electromagnetic emission. The malware must modulate the memory bus intensity to generate a signal with a sufficient Signal-to-Noise Ratio (SNR), enabling a nearby receiver to distinguish the data from environmental noise and accurately reconstruct the payload.

B. Victim Environment

The target environment is a secure facility housing an air-gapped system. The machine is physically disconnected from all types of networks through all the interfaces. The computer is running a standard modern operating system (e.g. Linux) and executing typical background tasks. The presence of other running processes introduces noise into the system (especially memory bound processes), which the modulation scheme must overcome. The system utilizes Double Data Rate (DDR) SD RAM. The vulnerability exploited is independent of the specific OS version and only depends on the physical design of the memory and the bus.

C. The Channel

The covert channel bridges the air gap through electromagnetic emanations. The channel utilizes the electromagnetic signals generated by the current fluctuations in the DDR memory bus. The attacker is positioned in close physical

proximity to the target system, typically within a range of some meters (depending on the signal strength the DDR memory bus produces). The attacker utilizes a passive receiver consisting of a portable Software Defined Radio (SDR) and an antenna. This setup allows the attacker to record and demodulate the signal without emitting any RF energy that could make them detectable.

IV. SYSTEM DESIGN: THE TRANSMITTER

A. Protocol Implementation

For the communication between the Transmitter and the Receiver, a simple custom communication protocol was implemented. Each data packet is preceded by a dedicated preamble byte, which serves as a synchronization marker and indicates the start of a valid transmission. Upon detecting this preamble, the receiver transitions into a reception state and begins decoding the incoming data stream. The end of a packet is identified by a termination pattern consisting of eight consecutive HIGH bits. When this bit sequence is detected, the receiver interprets it as the end-of-packet condition and finalizes the reception process. 1

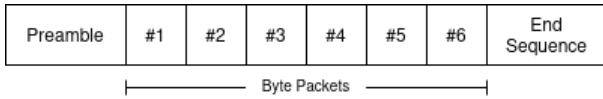


Fig. 1. Depiction of simple implementation packet.

This approach was used to test the communication between the SDR and the infected device, as it closely represents a realistic communication scenario. With this implementation, the data payload of a packet cannot include bit sequences that would be interpreted as either a start-of-packet preamble or an end-of-packet marker. For this reason, all testing was performed using carefully selected data sequences that satisfy this constraint and do not inadvertently trigger packet boundaries.

B. Future Protocol Designs

A more robust implementation would adopt a TLV (Type–Length–Value) encoding scheme [12]. In such a design, each packet explicitly encodes the type of the transmitted data (images, text files etc.) and the length of the payload before the payload itself, allowing packets of variable size to be transmitted without ambiguity. By relying on an explicit length field rather than a bit-pattern–based termination condition, this approach completely eliminates contention with end-of-packet markers embedded in the data stream.

Additionally, a TLV-based protocol simplifies packet parsing, improves extensibility by allowing new message types to be introduced without modifying the core protocol, and enables basic validation of packet integrity by checking length consistency.

This approach also enables straightforward extension of the packet format to include basic bit recovery and error-detection information, such as parity bits, checksums, or cyclic redundancy check (CRC) fields. By encoding these mechanisms

as additional TLV elements, error control information can be added or removed without impacting the core packet structure.

This modularity allows the communication protocol to be adapted to different channel conditions and reliability requirements. For example, simple parity may be sufficient for short-range or low-noise links, while more robust error-detection fields such as CRCs can be enabled for noisier environments.

C. Transmitter Implementation

The transmitter is implemented in C and relies on generating electromagnetic emissions from the memory bus to establish a covert channel. This is achieved by rapidly accessing memory in a way that bypasses the CPU cache, thereby maximizing activity on the bus.

The core of the emission generation is defined in the `steroids` function. It iterates through a large, pre-allocated buffer and uses the `_mm_stream_si128` intrinsic to perform non-temporal stores to a volatile register variable. This instruction directs the processor to write data directly to memory without caching, forcing high-frequency signals on the memory bus lines.

Modulation is implemented in `transmit_plain_bit` using a simple On-Off Keying (OOK) scheme. To transmit a logic '1', the system repeatedly invokes the `steroids` function for a duration defined by `PULSE_WIDTH`, generating a measurable activity on the memory bus. To transmit a logic '0', the system performs a busy-wait loop for the same duration, effectively silencing the memory bus emissions.

The transmitter is initialized by allocating a large memory buffer (`BUFFER_SIZE` set to 164,096 128-bit integers) and populating it with random values. This ensures that memory operations during transmission involve diverse data patterns, increasing the distinguishability of the OOK-modulated signal.

It is important to note that on older hardware, setting the buffer size to very large values can introduce timing issues. Specifically, memory operations may occasionally take longer than the selected bit period, potentially causing deviations from the intended bitrate and affecting signal integrity. Careful selection of buffer size and pulse width is therefore necessary to maintain accurate timing and reliable transmission.

D. Micro-architectural Implementation Details

To understand the generation of the side-channel signal, it is necessary to analyze the memory subsystem's behavior. In standard execution flows, memory writes are typically cached in the L1, L2, or L3 cache levels to minimize latency and energy consumption. This behavior is detrimental to the generation of electromagnetic emanations, as data movement confined within the CPU die generates negligible emissions compared to the electrical activity required to drive the long traces of the DDR memory bus.

The implementation overcomes this limitation by utilizing Non-Temporal Instructions (NTI). Specifically, the `_mm_stream_si128` intrinsic serves as a hint to the processor that the data being stored will not be reused in the near future. Consequently, the CPU bypasses the standard cache

hierarchy and writes the data directly to the Write-Combine Buffers (WCB) and subsequently to the main memory (RAM).

This architectural bypass effectively turns the memory bus traces on the motherboard into antennas. By forcing a continuous stream of data transfers to the main memory modules, the system induces rapid current changes along these traces, creating the electromagnetic radiation required for the covert channel. The use of random values in the buffer further ensures that the electrical switching activity is maximized, as opposed to writing static values (e.g., all zeros), which might result in lower power consumption and weaker signal strength.

E. Timing and Stability Analysis

Achieving a stable bitrate in a software-defined transmitter running in user space presents significant challenges related to timing determinism. Unlike a hardware-based radio (e.g., an FPGA or dedicated SDR) which operates on a strict clock cycle, this transmitter is subject to the scheduling policies of the host Operating System (OS).

The primary source of signal instability is process preemption and "jitter." Since the transmitter runs as a standard user-space process, the OS scheduler may interrupt the transmission loop to handle background tasks, system interrupts, or other high-priority threads. These interruptions effectively extend the duration of a pulse (logic '1') or a silence period (logic '0') beyond the intended `PULSE_WIDTH`. This phenomenon manifests as phase noise or timing jitter in the received signal, which can complicate synchronization at the receiver end.

To mitigate this, the implementation utilizes a "busy-wait" approach rather than standard sleep functions (e.g., `usleep`). System sleep calls typically have a coarse granularity (often limited to the OS tick rate, e.g., 1ms), which is insufficient for the microsecond-level precision required for high-speed OOK modulation. By actively burning CPU cycles in a tight loop to track time, the transmitter maintains finer control over the pulse width, albeit at the cost of higher CPU usage.

Furthermore, the size of the memory buffer (`BUFFER_SIZE`) introduces a trade-off between emission strength and timing stability. While a larger buffer forces more extensive memory scans and potentially stronger emissions, it increases the likelihood of triggering DRAM refresh cycles or page faults during a transmission burst. These hardware-level events stall the memory controller, introducing unpredictable latencies that distort the symbol period. Therefore, the buffer size must be tuned experimentally to find a "sweet spot" where emissions are detectable without inducing excessive timing drift.

V. SYSTEM DESIGN: THE RECEIVER

A. Receiver Implementation

The receiver was implemented using GNU Radio and is designed to parse the packets captured by the SDR antenna. The captured data stream is processed in real-time in cooperation with Python programs running on a third-party PC to which the SDR is connected. These programs are responsible

for decoding the packet contents and displaying the recovered message to the user.

This implementation was intentionally kept simple, focusing primarily on achieving high bitrates and maintaining correctness, as these were the primary objectives for a proof-of-concept demonstration. The receiver processes each incoming packet sequentially, detecting the preamble to identify the start of a transmission and monitoring the termination pattern to detect the end of the packet. Once a packet is fully captured, the Python programs handle the extraction of the payload and present it in a human-readable format.

Although functional, this approach requires careful tuning depending on the operational environment. Factors such as the distance between the infected PC and the SDR, ambient electromagnetic noise, antenna alignment, and the chosen bitrate all affect the reliability and accuracy of the reception. For example, higher bitrates may increase throughput but also make the system more sensitive to timing variations and noise, whereas lower bitrates may be more robust but reduce effective data transfer rates.

B. Future Receiver Designs

An alternative version of the receiver was implemented in an attempt to address the limitations of the previous manual thresholding approach, particularly regarding the need to manually tune threshold values for each experimental setup. This version incorporated an Automatic Gain Control (AGC) unit, which in principle should normalize the amplitude of the incoming data stream to a predefined range. By doing so, the differentiation between logic '0' and logic '1' would become trivial, eliminating the dependence on manually set thresholds and theoretically allowing the receiver to adapt dynamically to varying signal strengths or environmental conditions.

In practice, however, this AGC-based approach exhibited a poor Bit Error Rate (BER). While the AGC successfully adjusted the overall amplitude of the signal, it introduced additional variability and noise into the normalized waveform, which made reliable threshold detection difficult. Unlike the previous implementation that relied on fixed threshold parameters (`low_threshold` and `high_threshold`) tailored for each setup, the AGC version was intended to obviate the need for such tuning. Nevertheless, the resulting signal fluctuations and transient behavior caused misclassification of bits.

C. Digital Signal Processing (DSP) Chain

The demodulation process described above follows a non-coherent On-Off Keying (OOK) receiver architecture, implemented as a chain of interconnected signal processing blocks. In GNU Radio, these blocks are pre-generated and configurable functional units that process the incoming data stream sequentially. Each block performs a specific operation, such as filtering, thresholding, or packet detection, and passes the processed signal to the next stage in the chain.

Together, these blocks form a continuous data flow, ultimately producing the final `output.bin` file. This binary file contains the raw demodulated packet data and serves as input

for the subsequent Python scripts, which handle higher-level tasks such as packet parsing, data reconstruction, and message display.

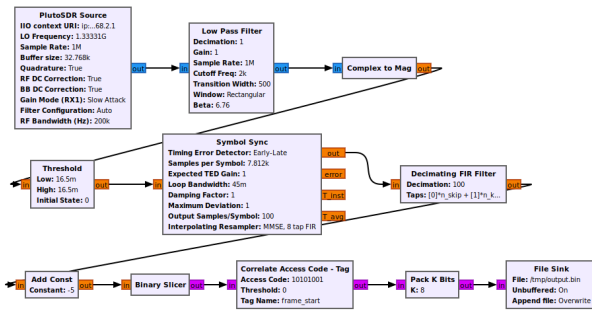


Fig. 2. GNU Radio DSP Chain.

The receiver processing chain consists of the following stages:

- **File Source:** The File Source acts as the entry point of the chain. Instead of capturing live signals from hardware, this block reads raw I/Q data from a pre-recorded file, ensuring repeatable testing and validation of the receiver. By replaying the same capture file consistently, experimental conditions are controlled, and algorithmic performance can be evaluated accurately.
- **Throttle:** The Throttle block limits the rate at which samples propagate through the flow graph. Without throttling, the CPU would process the data as quickly as possible, consuming excessive computational resources and generating outputs much faster than real-time. This would make visualization, analysis, and debugging impractical. By introducing a controlled throttle, the system enforces a “wall-clock” sample rate, effectively simulating live signal conditions and preserving correct timing relationships between samples. In practice, the Throttle block, along with the File Source, can later be bypassed when capturing data from a real-time SDR source, where the incoming samples inherently arrive at the correct physical rate.
- **Low Pass Filter:** The Low Pass Filter is applied to the stream. This FIR filter attenuates high-frequency noise while passing the narrowband OOK signal, which is centered at DC. By rejecting out-of-band interference, this stage cleans the signal and ensures that envelope detection operates on a clear representation of the transmitted waveform.
- **Complex to Magnitude:** The Complex to Magnitude block calculates the magnitude of the complex I/Q samples. This stage implements envelope detection, extracting the amplitude information that carries the OOK data. Phase information is discarded, as OOK encodes information solely in the presence or absence of a carrier.
- **Threshold:** The Threshold block converts the analog magnitude into a preliminary digital stream. By comparing the signal against defined high and low thresholds,

it produces a clean square-wave representation of the bitstream.

- **Symbol Synchronizer:** The Symbol Synchronizer recovers the symbol timing clock from the stream. This block identifies the optimal sampling points for each bit, correcting for timing jitter and drift between the transmitter and receiver clocks.
- **Decimating FIR Filter:** The Decimating FIR Filter implements a “Majority Vote” logic. It combines downsampling with a matched filter effect and effectively integrates the samples corresponding to the middle portion of each bit, ignoring noisy rising and falling edges. This improves BER by producing a single, reliable value per bit rather than selecting an arbitrary sample as the value of a bit.
- **Add Constant:** The Add Constant block shifts the range of the integrated signal to center bit values around zero for the final decision. Since the FIR output is strictly positive, subtracting half the maximum sum moves the decision boundary to zero, simplifying the binary slicing operation.
- **Binary Slicer:** The Binary Slicer converts the floating-point signal into a discrete bitstream. Values above zero are interpreted as logic ‘1’, while values at or below zero are interpreted as logic ‘0’. This finalizes the demodulation process, producing a clean digital representation of the transmitted data.
- **Correlate Access Code:** The Correlate Access Code block scans the bitstream for a predefined preamble sequence. Upon detecting the access code, it tags the stream to indicate the start of a packet, establishing framing for proper grouping of subsequent bits.
- **Pack K Bits:** The Pack K Bits block aggregates individual bits into byte-aligned words. By collecting 8 bits at a time, it produces standard 8-bit characters that can be written back to a regular text file.
- **File Sink:** The File Sink writes the processed byte stream to disk, producing the `output.bin` file. This file contains the fully demodulated payload and can be analyzed directly or used as input to external Python scripts for packet reconstruction, error checking, or message visualization.



Fig. 3. ADALM-PLUTO SDR.

By chaining these processing stages, the receiver implements a fully functional and modular OOK demodulator, capable of converting raw SDR captures of signals emitted by an infected host into usable binary data on a third-party PC.

VI. IMPLEMENTATION CHALLENGES

A. Overcoming the Cache Hierarchy

A fundamental challenge for any memory-based side-channel is the CPU's multi-level cache architecture. The Intel Celeron N2840 features a cache hierarchy (24 KiB L1 data cache and 1 MiB L2 cache) designed specifically to minimize external DRAM accesses. If the exfiltration malware is not carefully crafted, the target data remains within the silicon die, leaving the external memory bus silent and the covert channel inactive. To bypass this, our implementation utilized **Non-Temporal (NT) store instructions**. These instructions provide a hint to the memory controller to bypass the cache hierarchy and write data directly to the main DRAM. Implementing this in a way that remains efficient enough to generate a detectable "Logic 1" burst while avoiding CPU throttling was a major optimization challenge.

B. Mitigating System Jitter and Background Noise

The Linux Mint 20 environment used in our tests is a multi-tasking system that executes numerous background processes. This activity introduces significant "jitter" and unpredictable EM noise:

- **Temporal Jitter:** Variations in OS scheduling can cause the exfiltrated pulses to shift slightly in time, complicating symbol synchronization in GNU Radio.

- **DRAM Contention:** Other system processes accessing memory create erratic patterns that mimic exfiltrated data.

To mitigate this, we utilized a narrow-band *Low Pass Filter* in the receiver pipeline. However, at higher bitrates (e.g., 64–128 bps), the overlap between the exfiltration signal and environmental noise became more severe, leading to the failures at the 100 cm range.

C. Attacker-Induced Self-Interference

A significant and often overlooked challenge in electromagnetic exfiltration is the interference generated by the attacker's own equipment. During our experiments, we observed that the host device used for signal processing acted as a secondary, unintended source of EMI. Because the host system utilizes high-frequency DDR4 memory and complex internal power delivery systems, it generates a wide spectrum of harmonics. In several test runs, we identified parasitic harmonics originating from the attacker's notebook that were significantly close to the target 1.333 GHz frequency. Since the SDR is physically connected to the host via USB, the proximity of the antenna to the attacker's motherboard can lead to "self-jamming." This interference raises the noise floor and can mask the subtle bursts from the victim's memory bus. To mitigate this, we had to ensure a minimum physical separation between the SDR antenna and the host laptop. This challenge underscores the necessity of a "clean" spectral environment for high-bitrate exfiltration, even when the target is an isolated, air-gapped system.

D. Signal Attenuation and Distance Limits

The electromagnetic field strength of unintentional emanations decays rapidly as the distance from the source increases, following the inverse-square law. Our experiments confirmed that while the channel is highly stable in the near-field (25 cm), it becomes increasingly sensitive to environmental interference at a 100 cm range. The failure of the 64 bps bitrate at 100 cm demonstrated that there is a physical "ceiling" where the signal's amplitude envelope no longer provides enough contrast for reliable thresholding. This distance-dependent attenuation is a primary limiting factor for real-world exfiltration from air-gapped systems.

VII. EXPERIMENTAL EVALUATION

A. Experimental Setup

To empirically validate the memory bus electromagnetic exfiltration channel, we established a rigorous testbed consisting of a dedicated target victim and a sophisticated acquisition station. The physical configuration was designed to simulate a realistic air-gapped environment while allowing for precise measurement of bit error rates (BER).

1) *Target Device (Victim):* The victim system utilized for all experiments was a low-power laptop featuring an Intel Celeron N2840 System-on-Chip (SoC) based on the Silvermont (Bay Trail-M) architecture. This specific hardware was selected due to its out-of-order execution design and widespread use in entry-level computing, which provides a

representative noise floor for side-channel analysis. The system's detailed hardware specifications are as follows:

- **Processor:** Intel Celeron N2840 clocked at a base frequency of 2.16 GHz (burst up to 2.58 GHz), manufactured on a 22 nm production process.
- **Memory:** 2 GB of SODIMM DDR3 Synchronous memory operating at a bus frequency of 1333 MHz. The high-speed switching of this 1333 MT/s interface serves as the primary unintentional antenna for data exfiltration.
- **Cache Hierarchy:** A total of 1 MB L2 cache shared between cores, with 32 KiB L1 instruction cache and 24 KiB L1 data cache per core. These caches necessitated the use of non-temporal memory instructions to ensure direct bus activity.
- **Operating System:** Linux Mint 20 Cinnamon edition, configured to run standard background services to maintain a realistic environmental electromagnetic interference (EMI) profile.

2) *Acquisition System (Attacker):* The attacker utilized a high-performance MSI Modern 15 B12MO notebook as the primary processing host for the Software Defined Radio (SDR) data stream. The acquisition system was configured as follows:

- **Processing Host:** An MSI Modern 15 B12MO notebook powered by a 12th Gen Intel Core i5-1235U processor (10 cores, 12 threads, up to 4.4 GHz), ensuring sufficient computational throughput for real-time GNU Radio signal processing.
- **System Memory:** 16 GB (2×8 GB) of Samsung SODIMM DDR4 Synchronous memory operating at 3200 MHz, providing ample bandwidth for high-sample-rate I/Q data processing.
- **SDR Interface:** An ADALM-PLUTO (PlutoSDR) active learning module, chosen for its wide frequency range (325 MHz to 3.8 GHz) and ability to capture the 1.33331 GHz target frequency of the DDR3 memory bus.
- **Antenna Configuration:** A standard antenna tuned to the 1.3 GHz band was connected to the SDR. The antenna was placed at calibrated distances of 25 cm, 50 cm, and 100 cm from the target device's motherboard to evaluate signal attenuation.

3) *Measurement Environment:* All tests were conducted in a standard indoor office setting to account for typical 2.4 GHz Wi-Fi and Bluetooth interference. The target computer was physically isolated from all networks, with its wireless modules (Wi-Fi and Bluetooth) disabled at the kernel level to ensure that all exfiltrated data was carried solely via memory bus emanations.



Fig. 4. Image of the victim's system and the SDR, 100cm apart.

B. Results and Analysis

This subsection presents the empirical results of the exfiltration tests conducted using the setup described. The primary objective was to evaluate the reliability and throughput of the covert channel when transmitting a 128-bit cryptographic key across varying distances and bitrates.

1) *Transmission Duration:* The time required to exfiltrate a 128-bit key is inversely proportional to the selected bitrate. Table I details the theoretical and observed transmission windows for the 128-bit payload. Since the preamble and the end of message are negligible, we exclude them from the times.

TABLE I
TRANSMISSION TIME FOR 128-BIT KEY

Bitrate (bps)	Duration (Seconds)	Symbol Duration (ms)
2	64.0	500
4	32.0	250
8	16.0	125
16	8.0	62.5
32	4.0	31.25
64	2.0	15.625
128	1.0	7.8125

2) *Reliability and Bit Error Rate (BER):* The reliability of the exfiltration was measured by the Bit Error Rate (BER) across the testing matrix.

As shown in Table II, the channel maintained a 0% BER for all low-to-mid range bitrates (2–32 bps) at distances up to 100 cm. However, a significant BER was observed at the 64 bps bitrate at the 100 cm range.

3) *Analysis of Failures and Limitations:* The high BER of the 64 bps runs and failure of 128 bps runs highlights the physical limitations of the channel. As the distance increases,

TABLE II
OBSERVED BIT ERROR RATE (BER) (%)

Bitrate (bps)	25 cm	50 cm	100 cm
2	0%	0%	0%
4	0%	0%	0%
8	0%	0%	0%
16	0%	0%	0%
32	0%	0%	0%
64	9.67%	16.4%	29.68%
128	51.2%	failed	failed

the signal-to-noise ratio (SNR) degrades significantly. At 64 bps, the symbol duration is approximately 15.6 ms; at the 100 cm threshold, the “symbol smearing” effect—where the electromagnetic transitions of the memory bus bleed into adjacent symbols—prevented the GNU Radio *Threshold* and *Binary Slicer* blocks from maintaining a stable lock on the preamble.

VIII. CONCLUSION AND FUTURE WORK

A. Conclusion

This project successfully demonstrated that physical isolation is insufficient to prevent data exfiltration from modern computer systems. By exploiting the electromagnetic radiation emitted by the DDR memory bus, we established a covert channel capable of transmitting arbitrary data from an air-gapped machine to a nearby receiver.

Our implementation, consisting of the *ramear* malware [13] and a Software Defined Radio receiver, validated that user-space applications can manipulate hardware emanations without requiring administrative privileges or physical modifications. By utilizing non-temporal store instructions (such as `_mm_stream_si128`), we effectively bypassed the CPU cache hierarchy, forcing direct interaction with the main memory to generate detectable carrier waves. The experimental results confirm that these signals can be intercepted and demodulated using accessible hardware like the ADALM-PLUTO, undermining the core security assumption of air-gapped networks.

B. Future Work

While the current system proves the feasibility of the attack, several enhancements could improve the reliability, speed, and stealth of the covert channel.

1) *Protocol Optimization*: The current implementation relies on a sentinel-based framing protocol. The transmitter sends a specific 8-bit preamble (10101001) to mark the start of the transmission and a distinct 8-bit end-of-message (EOM) sequence (11111111) to terminate it. While simple to implement, this approach introduces a data transparency issue: if the binary payload naturally contains the EOM byte, the receiver will prematurely terminate the capture. Future iterations of the protocol should shift to a length-prefix framing structure. In this proposed design, the preamble would be immediately followed by a fixed-length header (e.g., 16 bits) representing the integer size of the payload. This would allow the receiver to

capture the exact number of expected bytes without scanning the payload for sentinel values. This modification would not only ensure data transparency but also improve demodulation efficiency, as the receiver could pre-allocate buffers and stop processing immediately upon receiving the final bit.

2) *Error Correction*: Currently, the transmission is vulnerable to environmental noise, and a single bit flip can corrupt an entire byte. To address this, Forward Error Correction (FEC) should be integrated into the modulation scheme. Implementing Hamming codes or Reed-Solomon error correction would allow the receiver to detect and mathematically correct bit errors caused by transient interference, significantly increasing the effective range and reliability of the channel.

3) *Stealth and Modulation*: The current On-Off Keying (OOK) modulation is effective but spectrally obvious. A dedicated spectrum analyzer would easily detect the rhythmic bursting of the memory bus energy. Future work could explore Frequency Shift Keying (FSK) by alternating between different memory access patterns that generate emanations at distinct frequencies, or Spread Spectrum techniques to distribute the signal energy across a wider bandwidth, making the transmission indistinguishable from the system’s natural background noise.

REFERENCES

- [1] Mordechai Guri et al. *Fansmitter: Acoustic Data Exfiltration from (Speakerless) Air-Gapped Computers*. 2016. arXiv: 1606.05915 [cs.CR]. URL: <https://arxiv.org/abs/1606.05915>.
- [2] Mordechai Guri et al. *BitWhisper: Covert Signaling Channel between Air-Gapped Computers using Thermal Manipulations*. 2015. arXiv: 1503.07919 [cs.CR]. URL: <https://arxiv.org/abs/1503.07919>.
- [3] Mordechai Guri, Boris Zadov, and Yuval Elovici. “LED-it-GO: Leaking (A Lot of) Data from Air-Gapped Computers via the (Small) Hard Drive LED”. In: *Detection of Intrusions and Malware, and Vulnerability Assessment*. Ed. by Michalis Polychronakis and Michael Meier. Cham: Springer International Publishing, 2017, pp. 161–184. ISBN: 978-3-319-60876-1.
- [4] Wim van Eck. “Electromagnetic radiation from video display units: An eavesdropping risk?” In: *Computers Security* 4.4 (1985), pp. 269–286. ISSN: 0167-4048. DOI: [https://doi.org/10.1016/0167-4048\(85\)90046-X](https://doi.org/10.1016/0167-4048(85)90046-X). URL: <https://www.sciencedirect.com/science/article/pii/016740488590046X>.
- [5] Wikipedia contributors. *Van Eck phreaking — Wikipedia, The Free Encyclopedia*. [Online; accessed 12-January-2026]. 2025. URL: https://en.wikipedia.org/wiki/Van_Eck_phreaking.
- [6] Markus G. Kuhn and Ross J. Anderson. “Soft Tempest: Hidden Data Transmission Using Electromagnetic Emanations”. In: *Information Hiding, Second International Workshop, Portland, Oregon, USA, April 14-17, 1998, Proceedings*. Ed. by David Aucsmith. Vol. 1525. Lecture Notes in Computer Science. Springer, 1998,

pp. 124–142. DOI: 10.1007/3-540-49380-8_10. URL: https://doi.org/10.1007/3-540-49380-8%5C_10.

- [7] Zhuoran Liu et al. “Screen Gleaning: A Screen Reading TEMPEST Attack on Mobile Devices Exploiting an Electromagnetic Side Channel”. In: *28th Annual Network and Distributed System Security Symposium, NDSS 2021*. 28th Annual Network and Distributed System Security Symposium, NDSS 2021. Publisher Copyright: © 2021 28th Annual Network and Distributed System Security Symposium, NDSS 2021. All Rights Reserved.; 28th Annual Network and Distributed System Security Symposium, NDSS 2021 ; Conference date: 21-02-2021 Through 25-02-2021. The Internet Society, 2021. DOI: 10.14722/ndss.2021.23021.
- [8] Mordechai Guri, Matan Monitz, and Yuval Elovici. “USBee: Air-gap covert-channel via electromagnetic emission from USB”. In: *2016 14th Annual Conference on Privacy, Security and Trust (PST)*. 2016, pp. 264–268. DOI: 10.1109/PST.2016.7906972.
- [9] Md Sadik Awal, Buddhipriya Gayanath, and Md Tauhidur Rahman. *Impedance vs. Power Side-channel Vulnerabilities: A Comparative Study*. 2024. arXiv: 2405.06242 [cs.CR]. URL: <https://arxiv.org/abs/2405.06242>.
- [10] Mohammad Sharawi and Malik Al-Qdah. “The design and simulation of a 400/533Mbps DDR-II SDRAM memory interconnect bus”. In: Aug. 2008, pp. 1–6. ISBN: 978-1-4244-2205-0. DOI: 10.1109/SSD.2008.4632797.
- [11] *ADALM-PLUTO Active Learning Module: Product Highlight*. Accessed: 2026-01-12. Analog Devices, Inc. 2017. URL: <https://www.analog.com/media/en/news-marketing-collateral/product-highlight/adalm-pluto-product-highlight.pdf>.
- [12] Wikipedia contributors. *Type-length-value* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 12-January-2026]. 2025. URL: <https://en.wikipedia.org/w/index.php?title=Type%E2%80%93length%E2%80%93value&oldid=1323783997>.
- [13] fyquah. *RAMEAR*. GitHub repository. PennApps Spring 2016 project. 2016. URL: <https://github.com/fyquah/ramear>.